LLM Architecture and Production Deployment

Introduction for Data Engineers

Data Engineering Boot Camp

July 01, 2025

Agenda

- Learning Outcomes
- LLM Architecture as Data Pipeline
- Production Integration Patterns
- Infrastructure and Operations
- Q&A and Discussion
- Next Steps

Learning Outcomes

By the end of this lesson, you will be able to:

- Describe the data flow through LLM architectures and draw parallels to data pipelines
- Identify integration patterns for incorporating LLMs into data systems
- Assess infrastructure requirements for LLM deployment
- Recognize operational considerations for monitoring LLM systems

Transformer Architecture Overview

Think of transformers as a sophisticated data processing pipeline, similar to ETL workflows.

Traditional ETL Pipeline

- Raw Data $\rightarrow$ Extract $\rightarrow$ Transform $\rightarrow$ Load $\rightarrow$ Query

LLM Processing Pipeline

- Raw Text $\rightarrow$ Tokenize $\rightarrow$ Embed $\rightarrow$ Process $\rightarrow$ Generate

Core Components

1. **Tokenization Layer**: Converts text to numerical tokens
2. **Embedding Layer**: Maps tokens to high-dimensional vectors
3. **Processing Layers**: Transformer blocks analyze relationships
4. **Output Layer**: Converts representations to human-readable text

Data Flow Example

Input: “The data pipeline processes millions of records daily”

1. **Tokenization**: [“The”, “data”, “pipeline”, ...] $\rightarrow$ [1234, 5678, 9012, ...]
2. **Embedding**: Token 1234 $\rightarrow$ [0.2, -0.8, 0.5, ...] (768D vector)
3. **Processing**: Attention mechanism computes token relationships
4. **Generation**: Predicts next token via probability distribution

Parallel Processing Advantages

Sequential (RNNs)

- Token 1 $\rightarrow$ Process $\rightarrow$ Token 2 $\rightarrow$ Process ...
- Slow, information loss
- Hard to parallelize

Parallel (Transformers)

- All Tokens $\rightarrow$ Process Simultaneously
- No bottleneck
- Highly parallelizable

Analogy: Sequential $\sim$ for-loop, Parallel $\sim$ Spark partitions

Training Pipeline

- **Pre-training**: Massive text $\rightarrow$ Base model
  - Data: Billions of pages (terabytes)
  - Cost: \$4–12M, months of GPUs
- **Fine-tuning**: Base model + domain data $\rightarrow$ Specialized model
  - Data: Domain-specific
  - Cost: \$10K–100K, days to weeks

Analogy: Pre-training $\sim$ data warehouse, Fine-tuning $\sim$ data marts

API-Based vs. Self-Hosted

API-Based (SaaS)

- App $\rightarrow$ API $\rightarrow$ LLM Service
- Pros: No infra, pay-per-use
- Cons: Privacy, rate limits
- Best for: Prototypes, moderate use

Self-Hosted

- App $\rightarrow$ Load Balancer $\rightarrow$ GPU Cluster
- Pros: Data control, no limits
- Cons: Complex, costly
- Best for: High volume, sensitive data

Batch vs. Real-Time Inference

Batch Processing

- Data Lake $\rightarrow$ LLM $\rightarrow$ Storage
- Uses: Document analysis, summarization
- Pros: Cost-efficient, high throughput

Real-Time Inference

- Request $\rightarrow$ Model $\rightarrow$ Response
- Uses: Chatbots, code completion
- Needs: Low latency, auto-scaling

Caching Strategies

- **Response Caching**: Cache frequent queries (Redis, CDN)
- **Model Caching**: Keep models in memory, multi-model serving
- **Context Caching**: Store conversation history, document embeddings

Integration with Data Infrastructure

- **Data Warehouse**: ETL $\rightarrow$ LLM $\rightarrow$ BI Tools
  - Example: Summarize customer feedback
- **Stream Processing**: Kafka $\rightarrow$ LLM $\rightarrow$ Analytics
  - Example: Real-time sentiment analysis
- **API Gateway**: Route NL queries to LLM, structured to DB

Infrastructure Requirements

- **Memory**: 7B model $\approx$ 14GB (FP16), +20–50% for inference
- **Compute**: GPUs essential, single GPU serves dozens of users
- **Storage**: 10–100GB/model, petabytes for training data

Cost Optimization

- **Model Selection**: Right-size, fine-tune smaller models
- **Infrastructure**: Spot instances, auto-scaling
- **Operational**: Batching, caching, load balancing

Monitoring and Observability

- **Performance**: Latency, throughput, resource usage
- **Quality**: Response quality, hallucination, bias
- **Infrastructure**: Server health, data pipeline, costs

Discussion Questions

1. Would you choose API-based or self-hosted LLM deployment? Why?
2. What processes could benefit from LLM integration?
3. How would you approach capacity planning for LLMs?

Key Takeaways

- LLM architecture mirrors data pipelines with parallel processing
- Integration patterns align with traditional architectures
- Infrastructure requirements are significant but manageable
- Operational considerations resemble high-scale systems

Preparation for Next Lesson

Topics:

- Technical and operational challenges with LLMs
- Ethical considerations and data governance
- Future implications for data engineering careers

Additional Resources

- **Architecture**:
  - Hugging Face Transformers
  - NVIDIA Triton Inference Server
  - Ray Serve
- **Examples**:
  - Netflix content recommendations
  - Spotify podcast transcription
  - Salesforce Einstein GPT